VKU · CROSS-PLATFORM MOBILE APP DEVELOPMENT

MINI-PROJECT
SHORT TECHNICAL REPORT

Course: Cross-Platform Mobile App Development (VKU)
Mini-Project Title: Mini-Project 2: Real-time Study Room Booking App
Team / Student Name: Nguyen Ngoc Bao Trung
Submission Date: 13/10/2026

1. GENERAL INFORMATION & DELIVERABLE LINKS

Team Members:

1. Nguyen Ngoc Bao Trung — **Student ID:** 23IT.B236
Role: Sole student / frontend, state management, booking logic and documentation
Contribution: 100% (sole submission)

Live Demo URL: Not provided; omitted at the student's request.

GitHub Repository: https://github.com/bbaotrungg/Mini-Project-2

Video Demo (Optional): Not supplied.

2. FEATURE IMPLEMENTATION CHECKLIST

#	Required Feature	Status	Implementation Details & Acceptance Level
1	Room discovery & status	**Complete**	8 sample rooms; bundled photos, building/floor, capacity and time-based Available Now / Occupied badges.
2	Multi-parameter filters	**Complete**	Instant search; A/B/C/V buildings; minimum capacity 2–20; all selected equipment must match.
3	Efficient room feed	**Implemented**	FlatList, memoized cards, fixed row measurements and bounded batches. Native 60fps is not measured.
4	Time slots & conflicts	**Complete**	7 days, 4 two-hour slots; booked/past slots disabled. Shared server rejects simultaneous duplicates.
5	Booking pass & check-in	**Complete**	UUID pass, interactive QR modal and self check-in from start minus 15 minutes until the session ends.
6	Zustand & persistence	**Complete**	useBookingStore owns session, reservations, filters and cancellation; AsyncStorage restores saved state.
7	Local reminders	**Implemented**	expo-notifications schedules at start minus 15 minutes; permissions, cancellation and tap routing. Phone delivery remains unverified.
8	Shared real-time updates	**Complete**	Socket.IO snapshots synchronize connected clients; server persists JSON and revalidates all booking actions.

3. TECHNICAL ARCHITECTURE & PROJECT STRUCTURE

The app uses Expo SDK 57, React Native, TypeScript, Zustand and AsyncStorage. Expo Router provides stack and tab navigation on top of React Navigation. A small optional Node.js server provides shared availability; the default local demo requires no server.

Project structure

```
Mini-Project-2/
├── README.md / package.json    Setup and root commands
├── study-room/
│   ├── src/app/                Screens and route layouts
│   ├── src/components/         RoomCard and BookingPass
│   ├── src/store/              useBookingStore
│   ├── src/services/           Real-time client and reminders
│   ├── src/core.ts             Shared booking/time/filter rules
│   ├── src/data/               Sample rooms and bundled photos
│   ├── server/                 Socket.IO server and JSON storage
│   └── tests/                  Rules, server and browser tests
└── submission/                 Report, screenshots and evidence
```

State management and booking flow

Screen selects room/slot → Zustand validates request → Local insert / server commit → Snapshot, pass and reminder

1. Hydration restores the student session, reservations, filters and reminder metadata before interactive screens render.
2. The discovery screen selects only the store fields it needs. Search, building, capacity and equipment filters combine immediately; cards are memoized.
3. Booking validates room, date, slot, start time and existing reservations. In local mode, validation and insertion run without an asynchronous gap.
4. In shared mode, the server performs authoritative validation and a synchronous disk commit before broadcasting the updated snapshot. Reconnection refreshes state; booking is blocked until synchronization succeeds.
5. A confirmed reservation receives a UUID. The app opens its QR pass and schedules a device-local reminder when permissions and timing allow.

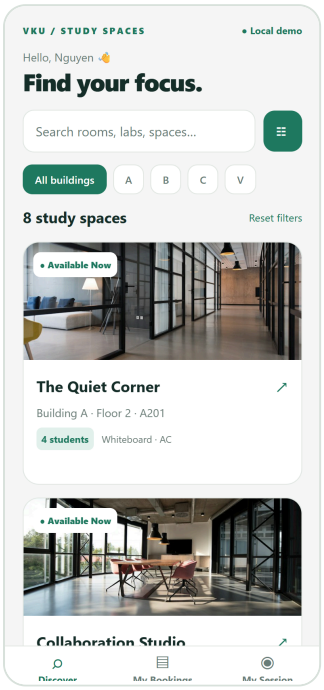
Exception handling and time consistency

- All slot calculations use Vietnam campus time (GMT+7). Midnight, year rollover, exact start/end boundaries and the seven-day window are covered by tests.
- Conflicts, disconnected sockets, request timeouts and ownership errors produce actionable messages. A busy guard prevents repeated booking submissions.
- Denied notification permissions and scheduling failures do not undo a valid booking. Web previews report reminders as unsupported.
- Cancellation releases the slot and removes the device reminder. Shared snapshots clean up reminders for remotely cancelled or checked-in bookings.

Scope: The server is a single-process classroom demonstration. Name-based sessions and self check-in are not production authentication. Multiple server instances would require transactional database constraints and verified student identity.

4. EMPIRICAL EVIDENCE & SCREENSHOTS

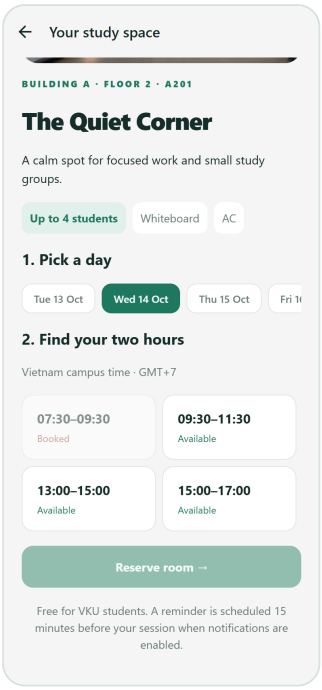
Evidence environment: These four screenshots show the running Expo web preview in Google Chrome at a 390 × 844 mobile viewport. They are browser evidence, not emulator or physical-device captures. The template's device screenshot requirement remains pending replacement with screenshots from Expo Go.



1

Discover rooms
Search, building chips and status badge are visible. Cards identify the building, floor, capacity and equipment.

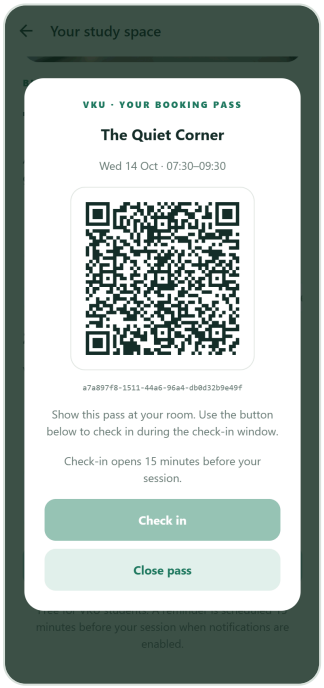
Acceptance: room photos load from bundled assets; no network image request is required.



2

Prevent conflicts
The 07:30–09:30 slot displays “Booked” and is disabled after reservation. Other future slots remain available.

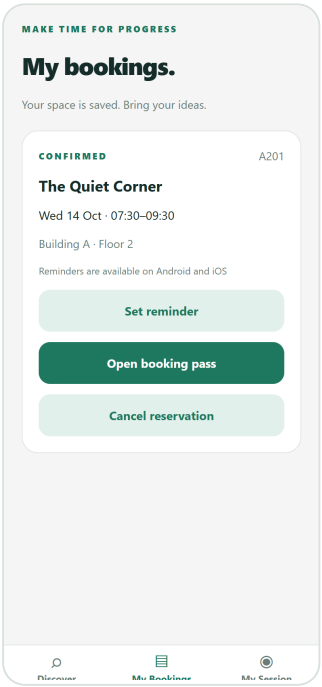
Acceptance: seven dates and discrete slots use GMT+7 campus time.



3

QR booking pass
The pass contains the room, date/time, QR payload and unique booking ID.

Acceptance: check-in is disabled before the window opens, and becomes valid at start minus 15 minutes.



4

Persisted reservation
The booking remains visible after a page reload. The student can reopen the pass or cancel.

Acceptance: web reminder limitations are stated visibly instead of reporting a false success.

Capture assertions: student session entry, loaded room photo, successful reservation, visible QR pass, disabled booked slot and persistence after reload. No browser runtime page errors were recorded. Native notification delivery and native frame-rate measurements are not evidenced by these captures.

5. TECHNICAL CHALLENGES & RESOLUTIONS

Challenge 1 — Concurrent reservations and stale availability

Disabling a slot in the interface alone cannot prevent two students from submitting the same booking before either sees the other's update. Persisted client data can also become stale during a disconnection.

Resolution: one server process owns the authoritative reservations. It validates the slot, writes the next snapshot to disk and broadcasts the result synchronously, without an await between conflict checking and insertion. The client blocks booking while disconnected or awaiting a fresh snapshot. It still revalidates locally to provide immediate feedback.

Evidence: an automated two-client race accepts exactly one reservation and rejects the other. Additional assertions verify cancellation releases the slot, another student cannot cancel it, restored server data still blocks a duplicate, and failed disk writes never acknowledge a booking.

Challenge 2 — Expo Go notification crash on Android

In the installed SDK 57 package, importing the top-level expo-notifications entry point eagerly loads remote push-token registration. That code throws in Android Expo Go, even though this app needs only local reminders.

Resolution: a dedicated localNotifications adapter imports only local scheduling, permission, channel, response and handler modules. Reminder functionality stays enabled. The adapter centralizes SDK-specific paths so future upgrades can be checked in one place.

Evidence: regression tests inspect the installed Android dependency graph and prohibit direct package-entry imports in app code. An exported Android source map includes local scheduling and channel modules and contains none of the crashing remote push modules. After this change, the student reported the project working in Expo Go.

Validation summary

Check	Observed result / acceptance boundary
TypeScript and ESLint	Both pass with the current project source.
Automated tests	10 tests pass: time/filter rules, notification import regression and shared server integration.
Browser acceptance flow	Session, combined filtering, booking, QR pass, persistence and cancellation pass.
Expo / bundle checks	Earlier Expo Doctor run: 21/21 checks passed. Android, iOS and web exports completed; Android notification graph rechecked after the fix.
Physical-device evidence	Student confirmed Expo Go works. Annotated device captures and actual reminder delivery are still pending.
Scrolling performance	List optimizations implemented; native 60fps is a target, not a measured result.

Reproduction and references

Clone the repository, run **npm install** then **npm start** from its root. On Windows PowerShell, use npm.cmd. For shared availability, follow the README's server and environment setup. Validation logs are included in submission/evidence.

Expo local notification support: docs.expo.dev/versions/v57.0.0/sdk/notifications/
Zustand persistence: zustand.docs.pmnd.rs/reference/integrations/persisting-store-data